

Training LLMs for Generating IEC 61131-3 Structured Text with Online Feedback

Aaron Haag*, Bertram Fuchs*, Altay Kacan*, Oliver Lohse*

*Siemens Technology Department, Munich, Germany

Email: {aaron.haag, bertram.fuchs, altay.kacan, oliver.lohse}@siemens.com

Abstract—IEC 61131-3 Structured Text (ST) is a widely used programming language for programmable logic controllers (PLCs) in automation systems. However, generating ST code with LLMs poses unique challenges due to limited data in public training datasets and the complexity of ST language syntax. This paper proposes an approach to fine-tune LLMs for the generation of ST code that leverages a preference-based learning method through an online process involving compiler feedback and evaluation from an LLM-based ST expert. In this framework, the model is iteratively refined and generates new training samples, which are subsequently evaluated by a compiler for syntactical correctness and by a specialized LLM that excels at assessing semantic accuracy, though it is not optimized for code generation itself. This approach results in marked improvements for the trained LLM, leading to higher compilation success rates and better semantic precision. As a result, the framework proves highly suitable for industrial automation applications and outperforms state-of-the-art models.

Index Terms—IEC 61131-3; LLMs; LLM Fine-tuning; Compiler feedback;

I. INTRODUCTION

We are entering a new era of code intelligence, where AI-driven copilots are transforming how developers interact with code. These tools, powered by Large Language Models (LLM), streamline complex programming tasks and significantly enhance productivity [1]. However, as we approach a potential depletion of high-quality internet data by 2026, the necessity of synthetic data for model training will increase [2]. This shift presents significant challenges, particularly for niche programming languages such as IEC 61131-3 Structured Text (ST), which is prevalent in Programmable Logic Controller (PLC) engineering and industrial automation systems [3]. To ensure that LLMs provide value in such specialized domains, it is essential that these models are aligned with expert expectations and values, optimizing their output for real-world needs [4], [5].

Reinforcement Learning from Human Feedback (RLHF) [6], a pioneering LLM alignment method, trains a reward model (RM) using pairwise human preferences and then optimizes the policy through reinforcement learning (RL) based on the RM. Recently, Direct Alignment from Preferences methods (DAP) have emerged as popular alternatives to RLHF [7], [8]. Unlike RLHF, DAP methods directly update the language model using pairwise preference data, simplifying the alignment process because there is no need to train a dedicated reward model. In the original DAP approach [9], preference datasets are gathered prior to training and are

based on expert feedback. With LLMs growing in size and complexity, this method can become prohibitively expensive if human experts label IEC 61131-3 conform generated code. Therefore, innovative alignment methods beyond direct human feedback are needed to fully leverage these models.

In response to this challenge, this work utilizes synthetic experts in an iterative online approach based on syntactic compiler feedback and a dedicated semantic LLM expert. To overcome the data scarcity of PLC code samples, the proposed framework exclusively exploits preexisting user prompt and code pairs derived from the APPS dataset [10] for SFT, as well as additional prompts that are subsequently used in an online finetuning scheme similar to Online AI Feedback (OAIF) [11]. This eliminates the need for large code datasets. The approach iteratively refines the model and incorporates feedback on both syntactic and semantic correctness, ensuring progressively better outcomes.

We base our proposed method on three concepts to overcome data scarcity in the ST code generation domain. First, there exists a performance margin between code generation and code quality judgement. Datasets used for training State-of-the-Art GPTs include only a small number of ST code samples, or none at all. Even though these models may not be able to generate ST code themselves, they are able to judge ST code samples due to semantic similarities with other languages like Pascal, which they were trained on extensively. Similarly to concepts like OAIF [11] or [12] we use the margin between generation and evaluation performance of SOTA LLMs to enhance ST code generation. While DPO typically applies to static offline preference use, recent studies found that DPO also benefits from iterative online training [13]–[15].

Second, using compiler feedback allows an accurate mapping from a diverse distribution of code samples to a binary compile label for each intent input [16]. This potentially enables training on an infinite amount of synthetically generated and syntactically flawless code samples beyond the syntax judgment performance of LLM judges.

Third, leaning on the concept of natural selection, a set of many code samples is generated instead of only code sample pairs. This enables the syntactic (compiler feedback) and semantic judge (LLM) to explore a broader distribution of samples [17]. It was shown that a high number of samples results in improved output quality [17], [18]. Hence, we believe that a sufficient number of code samples has to be generated to find a syntactically and semantically flawless code

sample.

The remainder of this paper is organized as follows: Section II reviews state-of-the-art methods. Section III introduces the proposed method. Section IV presents simulation results and discusses the future potential of the approach. Finally, Section V concludes the paper.

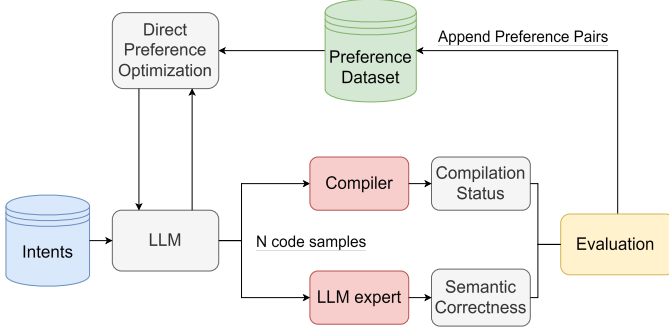


Figure 1: Overview of the DPO fine-tuning architecture, using a combination of feedback from a compiler and LLM experts.

II. STATE-OF-THE-ART METHODS

Before giving details on the proposed methodology, we investigate state-of-the-art methods for LLMs for code generation, OAIF and compiler feedback.

A. LLM-based Code Generation

Initial approaches for LLM-based code generation took problem statement prompts as input and directly generated code outputs conditioned on the prompts [19]. More recent techniques, such as chain-of-thought prompting [20] focus on breaking down tasks into modular or pseudo-code steps to improve planning and minimize errors. Furthermore, retrieval-based strategies incorporate similar problem samples and code solutions to guide the code generation process [21]. While these objectives are effective for natural language tasks, they fall short in fully capturing the complex syntactic and semantic rules of programming languages, often resulting in issues like syntax errors and type mismatches. Additionally, the success of these models in code generation is hindered by the need for large domain-specific datasets, which are scarce for specialized fields like industrial PLCs. As a result, LLM-generated code often fails to pass test cases and lacks built-in mechanisms for automatic detection and correction of syntactic and semantic errors [22].

B. Recent advances in LLM Alignment

Generative Pre-trained Transformer (GPT) models show great potential to generate PLC code based on domain-specific commands [23]. However, a significant challenge in this area remains the limited availability of data, which complicates effective model training and hinders performance. To address these challenges, the LLM4PLC project fine-tuned an open source model using publicly available PLC libraries such as OSCAT, demonstrating significant improvements in PLC code

generation [24]. Despite recent advances, models still struggle with first-attempt code accuracy, typically requiring multiple iterations to resolve compilation errors. For effective use of code in industrial applications, further refinements are needed.

An approach that greatly improves code quality is Reinforcement Learning for Human Feedback (RLHF). In RLHF, human evaluators provide preference data by comparing pairs of model outputs. This data is used to train a reward model (RM), which is incorporated into the LLM model training process [5]. Recently, DAP methods have gained popularity as alternatives to RLHF. DAP methods, such as Direct Preference Optimization (DPO) [9], update the target policy directly from preference pairs by training models with a preference model that simulates the evaluation of human feedback. In real-time, labelers compare model outputs to ensure consistency, but gathering large-scale annotations from human labelers is both expensive and logistically challenging. In most real-world applications, due to the high cost and complexity of collecting pairwise preferences, the preference dataset is typically compiled before aligning a language model and remains fixed throughout training. Gathering real-time preferences on new responses is typically impractical due to the absence of a human-in-the-loop for continuous feedback. As a result, the fixed dataset makes all preference data static, preventing the policy from receiving feedback on its own generated output during the alignment process. Relevant works, such as Coarse-Tuning [25], CodeRL [26], StepCoder [27], and MapCoder [28] utilize compiler feedback to improve model training. Recent advances use DPO iteratively, e.g. [9] [13].

C. LLM-based Structured Text Code Generation

Structured Text is one of the programming languages defined in the standard IEC 61131-3 for programmable logic controllers and has been designed for automation and control tasks. As the necessity of LLMs able to generate ST code is prevalent, several groups have developed methods for ST code generation, recently. Multiple base LLM models like GPT-3 and GPT-J have already been finetuned for generating ladder logic, function block diagrams, and ST code [29]. The models were found to generate basic PLC code, but struggled with more complex logic and integration with real-world PLC hardware and libraries. Another group showed that retrieval-augmented ST code generation integrates function blocks from libraries like OSCAT into the LLM-generated code [30]. LLM4PLC further introduced syntax checkers and model verifiers and utilized fine-tuning with Low-Rank Adaptations (LoRAs) to adapt LLMs for ST code generation. The applied methods showed a significant improvement in ST code generation performance, evaluated by domain experts [31]. The summarized papers were all published in 2024 and demonstrate the growing industrial interest for ST code generation. Despite these efforts, improvements are required to meet industry requirements and shrink the margin between finetuned models for ST code and SOTA models for frequently used programming languages like Python.

III. METHOD

This section describes the two subsequent stages for fine-tuning the LLM with special emphasis on the online-dpo method.

A. Supervised Fine-Tuning

Supervised Fine-Tuning (SFT) adapts a pre-trained language model (LLM) θ_0 for domain-specific tasks. During SFT, the model is trained on a set of instruction-response pairs $D_{\text{sft}} = \{(d_1, d_1^r), \dots, (d_m, d_m^r)\}$. This step ensures the model can follow instructions accurately and apply specific syntax, such as ST. SFT bridges the gap between general language understanding from pre-training and the specialized requirements of downstream tasks by maximizing the likelihood of generating the correct response d^r (i.e. ST control code) given the instruction d (i.e. code intent), as described in Equation 1.

$$\mathcal{L}_{\text{SFT}} = \arg \max_{\theta_0} \sum_{(x,y) \in D_{\text{sft}}} \log \pi(y|x) \quad (1)$$

SFT is conducted as the first step of fine-tuning, using an instruction-response dataset consisting of user intents and corresponding code. This assumes that a sufficient number of pairs is available. For ST, there are only a few public datasets (e.g., OSCAT [32]). The synthetic generation of such datasets can be time-consuming and expensive, especially if the model has limited understanding of code.

B. Direct-Preference-Optimization with AI and Compiler-Feedback

DPO (Direct Preference Optimization) is a widely used technique for model alignment based on expert feedback. In contrast to SFT (Supervised Fine-Tuning), it utilizes an instruction and preference triple rather than an instruction-response pair. This means that instead of requiring a code example, it only needs to be determined which output from a model, based on a given prompt, is preferred over another. This approach has the advantage of exploiting formal evaluation methods, such as unit tests for code as an expert. An actual reference code sample is not needed. Classical offline DPO uses static preference datasets and conducts a single training round on them [19]. These datasets can be generated in various ways, such as by querying a better model (a form of knowledge extraction) or by using the model being trained. If the model lacks sufficient performance, sampling methods or self-correction approaches based on feedback could be employed. However, this has the drawback of requiring a large number of samples to obtain high-quality positive preference pairs, or it may lead to a distribution shift between the preference samples and the actual model, which could result in degraded learning performance. To achieve continuous improvement of the model, we employ an iterative online variant of DPO [33]. This newer DPO training technique ensures that the preference pairs remain aligned with the model's distribution, also allowing the model to benefit from its own improvements in generating positive samples. In this approach, the model being

aligned generates samples, which are then used as preference pairs for fine-tuning through DPO. This process is repeated iteratively: the updated model generates new outputs, which are again evaluated by the expert to create new preference samples for further fine-tuning. This has the advantage that only instructions and an expert for the preference collection are needed for training, rather than instruction-response pairs with code, which are often scarce in the case of SFT for code.

C. Description of preference collection

The procedure can be described formally as follows.

Let the instruction set be defined as: $\mathcal{X} \triangleq \{x_1, x_2, \dots, x_M\}$ where $M = |\mathcal{X}|$ denotes the cardinality of the set.

For each iteration $i \in \{1, \dots, I\}$, a subset $X_i \triangleq \{X_{i,1}, \dots, X_{i,N}\}$ is drawn from $\mathcal{X}$: $X_i \sim \pi_i(\mathcal{X})$, where $\pi_i(\mathbf{X})$ represents the selection strategy applied during iteration i and N is the number of drawn instructions. For each instruction, a number of T code responses from the model θ_i is sampled, the set is denoted by $Y_i \triangleq \{Y_{i,1}, \dots, Y_{i,T}\}$. In this paper, two experts are queried to build preferences over the responses. The first expert is the compiler, which checks the syntactic correctness. The second expert is an LLM, which is used as a soft substitute for unit tests. The experts are combined to generate the preference pairs in the following way. Each generated code sample in Y_i is tested on the compilation. The samples that compile are checked by the semantic expert using a LLM. The samples that do not compile are treated as negative samples. The LLM as semantic expert then labels the compiling samples again as positive or negative, depending on their semantics. All positive samples are then combined with all negative samples. In case no samples are positive after the compile check, all samples are given to the LLM and labeled.

The combination of the respective expert feedback ultimately results in a preference dataset D_i , where $Y_w \succ Y_l$:

$$D_i = \{(X_{i,n}, y_{w,n}, y_{l,n})\} \forall n \in \{1, \dots, N\},$$

D. Direct-Preference-Optimization Fine-Tuning

After collecting the preference dataset D_i in the i th iteration, a model training leveraging DPO is conducted and the model parameters are updated to θ_{i+1} . The overall process is illustrated in Fig. 1.

IV. SIMULATION RESULTS AND DISCUSSION

This section presents the simulation results that illustrate the effectiveness of the proposed method.

A. Performance Metrics

To quantify the performance of the method, we define evaluation metrics appropriate for the ST experts which are abbreviated from [31] [19].

- **Compilation rate, P_c** – The empirical probability that a code segment C_i successfully compiles. This metric

Algorithm 1 Fine-Tuning algorithm to train a LLM for ST code generation

Input:

- Number of training iterations I
 - Instruction dataset $\mathcal{X}$
 - Supervised dataset $\mathcal{D}_{\text{sft}}$
 - Code generation model θ_0
 - Structured Text experts (semantic and syntax experts)
- 1: Initialize θ with supervised fine-tuning (SFT) to get θ_0
 - 2: **for** $i \leftarrow 1$ to I **do**
 - 3: Sample N ST intents $X_i \sim \pi_i(\mathcal{X})$
 - 4: Generate T ST code responses from θ_{i-1} for each intent
 - 5: Leverage expert feedback to construct preference dataset D_i
 - 6: Update model θ_i using D_i via DPO
 - 7: **end for**

Output: Aligned code generation model θ_I

reflects the likelihood that the code passes syntax and type-checking stages without errors.

- **Semantic correctness rate**, P_s – The empirical probability that generated code is semantically accurate. The primary criterion is the quality of the implementation and its alignment with the intent.
- **Joint semantic correctness and compilation rate**, P_j – The empirical joint probability that a generated code sample is both syntactically valid and semantically correct. This metric reflects the overall success in passing both compilation and semantic validation processes.

These customized metrics provide evaluation for the quantitative aspects of the method.

B. Experimental Setup

Parameter	Description
Base Model	$\theta_0 \sim \text{Phi-3-medium-instruct LLM}$
Selection Strategy	$\pi_i \sim \mathcal{U}(\frac{1}{M}) \forall i \in \{1, \dots, I\}$
Supervised Dataset ($\mathcal{D}_{\text{SFT}}$)	300 OSCAT, 1000 converted APPS
Instruction Set ($\mathcal{X}$)	100 OSCAT, 900 converted APPS intents
Number of code responses	$T = 15$
Iterations	$I = 9$
Evaluation Dataset	100 OSCAT, 100 converted APPS intents
Semantic ST Expert	gpt-3.5-turbo-1106 + prompt (Fig. 2)
Syntactic ST Expert	Rusty compiler [34]
Evaluators	Rusty compiler + gpt-4-1106-preview

Table I: Experimental Parameters for the Study

The baseline model θ is based on the Phi-3 LLM architecture [35], specifically the version with 14 billion parameters (Phi-3 14B). To maintain its optimal performance, no quantization techniques were applied. The model was reparameterized using SFT with a synthetic dataset generated in advance. This dataset was derived from the APPS dataset [10], which contains prompt-code pairs. The implementations were

SYSTEM: “You are an expert in PLC Structured Text (ST) programming as per IEC 61131-3 standards. Your task is to evaluate the given code based on its logical, semantic, and syntactic correctness. You can tolerate minor mistakes or errors.

IMPORTANT: Use the following format for your responses: “[0] [1] [1]”, where 0 indicates a negative evaluation and 1 indicates a positive evaluation.

IMPORTANT: Ensure that the number of brackets matches the number of given implementations.

IMPORTANT: Provide the evaluations strictly in the specified format without any additional text, code, or formatting.

INSTRUCTION: Given implementations are separated by ‘=====’.

USER: “Great! Here is another problem and its corresponding implementations to evaluate.

Problem: $X_{i,n}$

Implementations: subset of Y_i that compiles or Y_i if no samples compile”

Figure 2: The expert prompt consists contains the specific intent $X_{i,n}$, along with generated code samples from Y_i and an ST expert instruction.

transpiled from Python to ST codes using GPT-4, with filtering applied to exclude examples that were either too complex algorithmically or too specific to Python. This conversion process resulted in approximately 2000 ST code samples, which were subsequently used for SFT. During the experiments, Parameter-Efficient Fine-Tuning (PEFT) [36] was employed as fine-tuning approach, with Low-Rank Adaptation (LoRA) [37] used to fine-tune the language model via DPO. This configuration was chosen to optimize performance by minimizing the number of trainable parameters, significantly reducing computational resource consumption. The LoRA modules were integrated into the transformer layers of the model, enabling efficient adaptation while preserving the underlying pre-trained knowledge and avoiding significant deviation from the baseline model.

The experiments were conducted using the Phi-3 model on two NVIDIA GTX A6000 GPUs, providing a total of 96GB of memory.

C. IEC 61131-3 ST Experts

There is a limited availability of compilers compliant with the IEC 61131-3 standard [38] for the ST language. The syntactic expert is selected based on the capabilities of the open-source compiler Rusty [34]. Rusty offers the essential features required for syntactical analysis and facilitated the integration of this expert into the overall framework. This expert returns a binary feedback on the compilation success. As semantic expert ChatGPT-3.5 is used and queried for a binary evaluation of given ST code. For the evaluation as

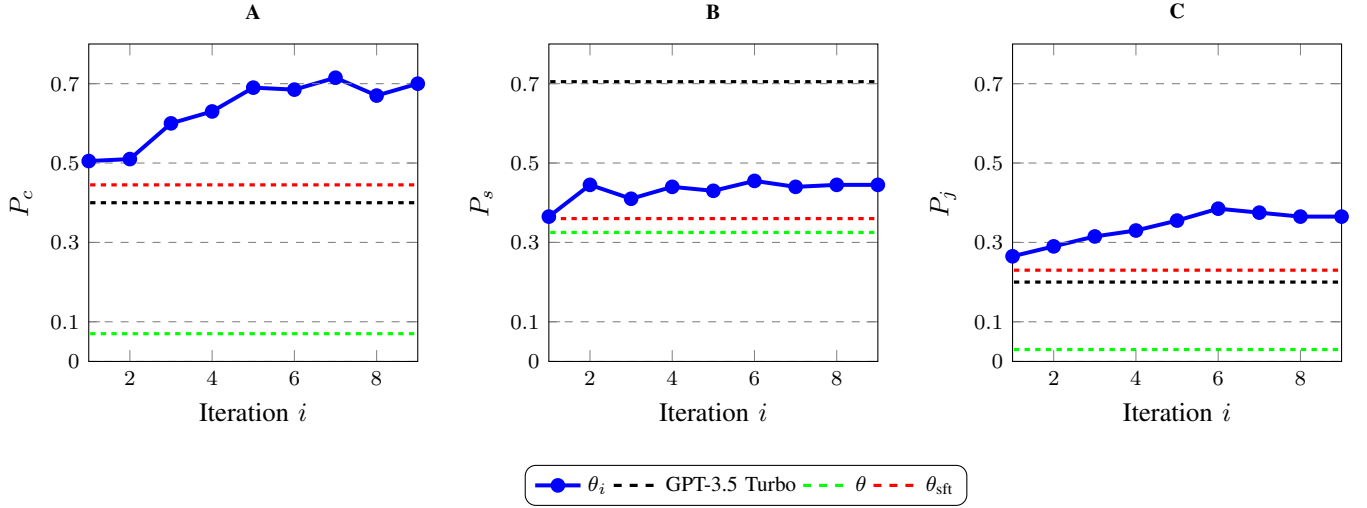


Figure 3: Improvement of different metrics over iterations: **A.** Compilation rate improvement across training iterations. **B.** Semantic correctness rate improvement across training iterations. **C.** Joint improvement of both compilation and semantic correctness rates over training iterations.

shown in Fig. 3, the semantic expert is GPT-4. This setup aims to allow for cross-validation between different GPT versions, ensuring that one model was not evaluating its own performance. Both models were provided with the same expert prompt as shown in Fig. 2, incorporating an evaluation prompt and corresponding original intent for the generated code to be evaluated.

D. Results and Discussion

The following paragraphs describe the key performance indicators used to assess the efficacy of the proposed method, focusing on the improvements in both the compilation success rate and semantic correctness of the generated ST code.

The graphs in Fig. 3 break down the improvements across number of $I = 9$ training iterations. Also, the performance of the base model θ , as well as the STF variant θ_{sft} is shown. As a closed-source competitor ChatGPT-3.5 is shown. From Iteration 1 to 9, we observe the performance following the application of DPO. As depicted, there is a notable upward trend in the performance metrics, with the most significant improvements occurring after Iteration 2. After that, the metrics plateau, indicating no further expected improvements.

As illustrated in Fig. 3A, the results demonstrate a significant enhancement in the compilation success rate compared to the baseline model. The proposed method achieves a compilation success rate of around 70%, a substantial increase from the baseline's 7%. This also surpasses the performance of GPT-3.5, which yielded a 40% compile rate. Although feedback in DPO is provided in binary form rather than through multiple reward values based on compilation status, it seems like the DPO loss function effectively compensates for this by utilizing its internal reward mechanism.

As shown in Fig. 3B, the model's semantic correctness rate peaked at around 45%. This improvement was facilitated by the integration of semantic expert represented by ChatGPT-3.5, which enabled the implementation of a ranking policy

through its textual comparison capabilities. The expert's ability to evaluate each code sample $\hat{C}_i$ contributed to the refinement of the model and increase in both syntactic and semantic correctness. This suggests that DPO is not only effective in natural language tasks but is also highly efficient in programming code generation that requires strict adherence to syntactic and semantic rules.

The joint compilation and semantic correctness rate depicted in Fig. 3C of approximately 39% was achieved after multiple iterations. This metric highlights the reliability and applicability of the generated ST code in industrial automation, where both accuracy and functionality are essential. The iterative process of training, evaluation, and feedback was critical to achieving these results.

Overall, substantial improvements have been demonstrated when comparing this approach to baseline models. The improvements are shown to be continuous over iterations.

This suggests that further refining feedback loops and data collection processes could lead to better performance gains. One potential area for further development involves the use of stronger baseline models. While Phi-3 14B performs well in many natural language processing and reasoning tasks, it may encounter challenges with more advanced logic, complex algorithms, or generalization to intricate intents. Future exploration could address these challenges, improving its ability to handle complex tasks. LoRA efficiently fine-tunes models using fewer parameters, but this reduced parameter count may limit the model's ability to capture the full depth of certain programming languages, especially those it hasn't been deeply trained on. Future efforts could combine LoRA with other fine-tuning techniques to better capture the complex syntactic and contextual details needed for high-quality code generation. When evaluating code quality, the model relies on patterns learned from training data, which may not always cover edge cases or specific application needs. This emphasizes the importance of incorporating software testing methods, such

as unit and functional testing, to complement the model's evaluations.

Finally, the present work considers only PLC programming in the ST language; however, it will be interesting to explore Structured Text (ST) due to its unique syntactic differences that could benefit from further refinement.

V. CONCLUSION

The methodology presented in this paper employs an iterative approach to fine-tune a LLM for generation of Structured Text by using a supervised-fine-tuning process on a small dataset and iterative DPO based on a compiler and LLM expert. The integration of the compiler feedback and the LLM expert in form of a semantic correctness feedback leads to better results than the SFT model. The use of synthetic data and a preference-based fine-tuning process specifically addresses the data scarcity associated with domain-specific programming languages, making it possible to train LLMs without extensive data collection or costly human labeling steps. This approach enables the development of industry-specific copilots that can revolutionize how PLCs are programmed, paving the way for further industrial automation.

REFERENCES

- [1] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," 2022. [Online]. Available: <https://arxiv.org/abs/2206.15000>
- [2] R. Liu, J. Wei, F. Liu, C. Si, Y. Zhang, J. Rao, S. Zheng, D. Peng, D. Yang, D. Zhou, and A. M. Dai, "Best practices and lessons learned on synthetic data," 2024. [Online]. Available: <https://arxiv.org/abs/2404.07503>
- [3] "Siemens simotion documentation for structured text (st)." [Online]. Available: https://cache.industry.siemens.com/dl/files/879/98990879/att_64192/v1/ST_Programming_en-US.pdf
- [4] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang, "Sparks of artificial general intelligence: Early experiments with gpt-4," 2023. [Online]. Available: <https://arxiv.org/abs/2303.12712>
- [5] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>
- [6] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03741>
- [7] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," 2024. [Online]. Available: <https://arxiv.org/abs/2305.18290>
- [8] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, "Kto: Model alignment as prospect theoretic optimization," 2024. [Online]. Available: <https://arxiv.org/abs/2402.01306>
- [9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," 2023. [Online]. Available: <https://arxiv.org/abs/2305.18290>
- [10] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with apps," 2021. [Online]. Available: <https://arxiv.org/abs/2105.09938>
- [11] S. Guo, B. Zhang, T. Liu, T. Liu, M. Khalman, F. Llinares, A. Rame, T. Mesnard, Y. Zhao, B. Piot *et al.*, "Direct language model alignment from online ai feedback," *arXiv preprint arXiv:2402.04792*, 2024.
- [12] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu, "Self-play fine-tuning converts weak language models to strong language models," 2024. [Online]. Available: <https://arxiv.org/abs/2401.01335>
- [13] W. Xiong, H. Dong, C. Ye, Z. Wang, H. Zhong, H. Ji, N. Jiang, and T. Zhang, "Iterative preference learning from human feedback: Bridging theory and practice for rlhf under kl-constraint," in *Forty-first International Conference on Machine Learning*, 2024.
- [14] W. Yuan, R. Y. Pang, K. Cho, S. Sukhbaatar, J. Xu, and J. Weston, "Self-rewarding language models," *arXiv preprint arXiv:2401.10020*, 2024.
- [15] S. Xu, W. Fu, J. Gao, W. Ye, W. Liu, Z. Mei, G. Wang, C. Yu, and Y. Wu, "Is dpo superior to ppo for llm alignment? a comprehensive study," *arXiv preprint arXiv:2404.10719*, 2024.
- [16] X. Wang, Y. Wang, Y. Wan, F. Mi, Y. Li, P. Zhou, J. Liu, H. Wu, X. Jiang, and Q. Liu, "Compilable neural code generation with compiler feedback," *arXiv preprint arXiv:2203.05132*, 2022.
- [17] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large language monkeys: Scaling inference compute with repeated sampling," *arXiv preprint arXiv:2407.21787*, 2024.
- [18] M. Hassid, T. Remez, J. Gehring, R. Schwartz, and Y. Adi, "The larger the better? improved llm code-generation via budget reallocation," *arXiv preprint arXiv:2404.00725*, 2024.
- [19] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [21] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," 2021. [Online]. Available: <https://arxiv.org/abs/2108.11601>
- [22] T. Ridnik, D. Kreda, and I. Friedman, "Code generation with alphacodium: From prompt engineering to flow engineering," 2024. [Online]. Available: <https://arxiv.org/abs/2401.08500>
- [23] H. Koziolok, S. Gruener, and V. Ashwal, "Chatgpt for plc/dcs control logic generation," in *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2023, pp. 1–8.
- [24] M. Fakhri, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "Llm4plc: Harnessing large language models for verifiable programming of plcs in industrial control systems," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '24. ACM, Apr. 2024. [Online]. Available: <http://dx.doi.org/10.1145/3639477.3639743>
- [25] A. Jain, C. Adiole, S. Chaudhuri, T. Reys, and C. Jermaine, "Coarse-tuning models of code with reinforcement learning feedback," 2023. [Online]. Available: <https://arxiv.org/abs/2305.18341>
- [26] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "Coderl: Mastering code generation through pretrained models and deep reinforcement learning," 2022. [Online]. Available: <https://arxiv.org/abs/2207.01780>
- [27] S. Dou, Y. Liu, H. Jia, L. Xiong, E. Zhou, W. Shen, J. Shan, C. Huang, X. Wang, X. Fan, Z. Xi, Y. Zhou, T. Ji, R. Zheng, Q. Zhang, X. Huang, and T. Gui, "Stepcoder: Improve code generation with reinforcement learning from compiler feedback," 2024. [Online]. Available: <https://arxiv.org/abs/2402.01391>
- [28] M. A. Islam, M. E. Ali, and M. R. Parvez, "Mapcoder: Multi-agent code generation for competitive problem solving," 2024. [Online]. Available: <https://arxiv.org/abs/2405.11403>
- [29] K. Tran, J. Zhang, J. Pfeiffer, A. Wortmann, and B. Wiesmayr, "Generating plc code with universal large language models," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2024, pp. 1–8.
- [30] H. Koziolok and A. Koziolok, "Llm-based control code generation using image recognition," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, 2024, pp. 38–45.
- [31] M. Fakhri, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "Llm4plc: Harnessing large language models for verifiable programming of plcs in industrial control systems," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024, pp. 192–203.
- [32] OSCAT, "Oscat basic," 2023, product description and specifications. [Online]. Available: <http://www.oscat.de/de/63-oscat-basic-321.html>

- [33] S. Guo, B. Zhang, T. Liu, T. Liu, M. Khalman, F. Llinares, A. Rame, T. Mesnard, Y. Zhao, B. Piot, J. Ferret, and M. Blondel, "Direct language model alignment from online ai feedback," 2024. [Online]. Available: <https://arxiv.org/abs/2402.04792>
- [34] "Rusty compiler user documentation." [Online]. Available: <https://plc-lang.github.io/rusty/>
- [35] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [36] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment," 2023. [Online]. Available: <https://arxiv.org/abs/2312.12148>
- [37] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [38] *Programmable controllers – Part 3: Programming languages*, International Electrotechnical Commission Std. IEC 61 131-3:2013, 2013.